SANDIP UNIVERSITY
UGC Recognised

NAAC GRADE A

ML Network Intrusion Detector

A Machine-Learning Baseline for Classifying Network Traffic Anomalies

Project Report

Institution: Sandip University

Student name: _____

Department / Programme: _____

Course / Academic year: _____

Submission note

This report documents the implementation and evaluation procedure. Dataset-specific results are intentionally left to be populated after running the code; no experimental scores are fabricated.

Abstract

Network intrusion detection aims to identify traffic patterns associated with malicious or abnormal activity. This project presents an offline supervised-learning baseline implemented with Python and scikit-learn. It reads a labeled network-flow CSV, preprocesses numeric and categorical features, trains a class-weighted Random Forest classifier, evaluates predictions on a held-out test split, and saves both the trained pipeline and metrics. The framework is designed for reproducible educational experiments on public datasets. Its effectiveness depends on dataset quality, label definitions, feature selection, and the evaluation split; the implementation is not a live traffic monitor or a production security control.

1. Introduction

Modern networks generate large volumes of connection and flow records. Rule-based detection remains useful, while machine-learning methods can learn statistical relationships between observed features and known traffic labels. A supervised model can support analysis by assigning an input flow to one of the classes represented in its training data. However, unfamiliar attacks, shifting traffic patterns, imbalanced labels, and data leakage can reduce real-world reliability.

2. Problem statement

Develop a reproducible machine-learning pipeline that uses a labeled public network-traffic dataset to classify traffic records into benign/normal and attack-related classes, where those classes are present in the source labels. The program should preprocess data, train a model, calculate meaningful classification metrics, and export artifacts for later prediction.

3. Objectives

- Prepare numeric and categorical network-flow features for learning.
- Train a Random Forest baseline with class weighting to help address class imbalance.
- Evaluate performance using accuracy, weighted precision, weighted recall, weighted F1, and a confusion matrix.
- Save the model pipeline and machine-readable metrics.
- Provide a command-line interface for training and inference.

4. Tools and technologies

Python provides the implementation language. **pandas** and **NumPy** handle tabular and numerical data. **scikit-learn** supplies preprocessing, train/test splitting, the Random Forest classifier, and evaluation metrics. **joblib** serializes the trained pipeline.

5. Dataset

The code expects a labeled CSV file and does not bundle a dataset. Suitable public research datasets include CIC-IDS2017, CSE-CIC-IDS2018, UNSW-NB15, and NSL-KDD. The selected dataset, download source, version, license, and citation must be recorded by the project author. Dataset schemas vary, so the target column should be selected explicitly when it cannot be inferred. This report does not claim to have trained on any one dataset.

6. Methodology

1. Load the CSV and remove rows without a target label.
2. Replace infinite values with missing values and remove columns that are entirely missing.
3. Drop common identifiers such as flow IDs, IP addresses, and timestamps to reduce identity leakage.
4. Impute numeric values with the median; impute categorical values with the most frequent category.
5. Scale numeric features and one-hot encode categorical features within a scikit-learn pipeline.
6. Split data into training and test subsets, stratifying by class when possible.
7. Train a Random Forest with 250 trees and balanced class weights.
8. Evaluate on the held-out test split and save the model and metrics.

7. System design

CSV dataset → validation and label selection → feature cleaning → numeric/categorical preprocessing → train/test split → Random Forest classifier → evaluation report and saved model → predictions for new CSV records.

8. Implementation

The main implementation is in **src/network_intrusion_detector.py**. The **train** command fits and evaluates a complete preprocessing/classification pipeline. The **predict** command loads the saved pipeline and writes predictions for new records. The pipeline stores preprocessing together with the classifier so inference applies the same transformations used during training.

Example training command

```
python src/network_intrusion_detector.py train \
  --data data/network_flows.csv \
  --label-column Label
```

9. Evaluation plan and results

Run the program on the selected dataset and insert the generated values below. The program writes the full metrics to **artifacts/evaluation_metrics.json** and prints a classification report. Do not report unmeasured values.

Metric	Observed result (complete after execution)
Dataset name / version	_____
Label column and class definitions	_____
Rows / features used	_____
Accuracy	_____
Weighted precision	_____
Weighted recall	_____
Weighted F1-score	_____
Experiment date / environment	_____

Interpretation guidance: inspect class-wise recall and false negatives for attack classes, as well as false positives that may overwhelm an analyst. Accuracy alone is insufficient when benign and attack classes are imbalanced.

10. Limitations

• Performance is dataset-dependent and can be inflated by duplicates or leakage.
• A random split may not represent future traffic or a different network.
• The model can only predict classes represented during training.
• Predicted probabilities are not calibrated guarantees of correctness.
• The tool does not capture live packets, alert an operations team, or block connections.

11. Future enhancements

Possible extensions include time-based and host-based evaluation, cross-dataset validation, feature importance analysis, hyperparameter tuning, calibration, explainability, drift monitoring, and a carefully controlled integration with a live telemetry pipeline. Any deployment should include independent security review, privacy safeguards, and human oversight.

12. Conclusion

This project provides a clear baseline for supervised network-traffic classification using scikit-learn. It emphasizes a reusable preprocessing pipeline, class-aware evaluation, and exportable artifacts. Reliable conclusions require running the code against a documented dataset, reviewing class-wise errors, and testing for leakage and distribution shift. The current implementation is intended for learning and experimentation, not as a standalone security product.

References and dataset citations

1. scikit-learn documentation: https://scikit-learn.org/stable/
2. pandas documentation: https://pandas.pydata.org/docs/
3. CIC-IDS2017 dataset information: Canadian Institute for Cybersecurity, University of New Brunswick (cite the exact dataset page/version used).
4. UNSW-NB15 dataset information: UNSW Canberra Cyber (cite the exact dataset page/version used).

Replace or supplement these references with the exact dataset publication and access date for the experiment you perform.